

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 12. Sets

Lecture: 130.Set Methods - Union, Intersection, Difference etc

Section 1: Lecture Summary

This lecture explains **set operations in Python**, highlighting their direct correlation to mathematical set operations discussed previously. Python's built-in ``set`` data type supports operations like **union, intersection, difference, and symmetric difference**, along with their update variants that modify the original set instead of returning a new one. The lecture walks through each operation step-by-step, using two example sets to illustrate how these methods work, how they handle common and unique elements, and the difference between returning new sets versus updating existing ones.

Section 2: Key Concepts and Explanations

- Python Sets as Data Type

- Python's ``set`` type holds **distinct, unordered elements**.
- It is implemented as a class object with methods corresponding to mathematical set operations.

- Set Operations

- All operations can take an **iterable** (commonly another set) as an argument.
- There are two types of methods:
 - Those that **return a new set** with the result.
 - Those that **update the original set in place**.

- Union (`union(iterable)``)

- Combines elements from both sets, including all unique elements.
- Returns a new set containing all distinct elements from both sets.
- Example: If $S1 = \{1, 2, 3, 5, 7\}$ and $S2 = \{5, 7, 9, 10, 11\}$, then $S1.union(S2) = \{1, 2, 3, 5, 7, 9, 10, 11\}$.

- Intersection (`intersection(iterable)`)

- Produces a new set with only the **common elements** between both sets.
- Example: Intersection of $S1$ and $S2$ returns $\{5, 7\}$.

- Intersection Update (`intersection_update(iterable)`)

- Instead of returning a new set, it **updates the original set** by keeping only common elements.
- Called on $S1$ with $S2$ as an argument means $S1$ becomes the intersection result.

- Difference (`difference(iterable)`)

- Returns a new set containing elements from the original set **that are not in the given iterable**.
- For $S1.difference(S2)$, it returns elements present in $S1$ but not in $S2$.
- Example: $\{1, 2, 3\}$ for the given $S1$ and $S2$.

- Difference Update (`difference_update(iterable)`)

- Updates the original set by **removing elements found in the given iterable**.
- Modifies the set it is called on instead of creating a new one.
- E.g., `s1.difference_update(s2)` updates $s1$ to $\{1, 2, 3\}$.

- Symmetric Difference (`symmetric_difference(iterable)`)

- Returns a new set of elements that are **unique to each set** (elements in either set but not in both).

- For the example sets, symmetric difference is {1, 2, 3, 9, 10, 11}.
- **Symmetric Difference Update** (`symmetric_difference_update(iterable)`)
 - Updates the original set to the symmetric difference result; no new set is returned.

Section 3: Example Code and Use Cases

Define two sets

Union returns a new set with unique elements from both

Intersection returns a new set with common elements

Intersection update modifies s1 to keep only common elements

Reset sets for difference examples

Difference returns a new set with elements in s1 not in s2

Difference update modifies s1 removing elements found in s2

Reset sets for difference update on s2

s2 difference update with s1

Symmetric difference returns a new set of unique elements from both

Symmetric difference update modifies s1 to this unique set

```
s1 = {1, 2, 3, 5, 7}
s2 = {5, 7, 9, 10, 11}

s3 = s1.union(s2)
print(s3) # Output: {1, 2, 3, 5, 7, 9, 10, 11}

s3 = s1.intersection(s2)
print(s3) # Output: {5, 7}

s1.intersection_update(s2)
print(s1) # Output: {5, 7}

s1 = {1, 2, 3, 5, 7}
s2 = {5, 7, 9, 10, 11}
```

```

s3 = s1.difference(s2)
print(s3) # Output: {1, 2, 3}

s1.difference_update(s2)
print(s1) # Output: {1, 2, 3}

s1 = {1, 2, 3, 5, 7}
s2 = {5, 7, 9, 10, 11}

s2.difference_update(s1)
print(s2) # Output: {9, 10, 11}

s1 = {1, 2, 3, 5, 7}
s2 = {5, 7, 9, 10, 11}
s3 = s1.symmetric_difference(s2)
print(s3) # Output: {1, 2, 3, 9, 10, 11}

s1.symmetric_difference_update(s2)
print(s1) # Output: {1, 2, 3, 9, 10, 11}

```

These examples show how each method is used and the effect on the sets involved—whether a new set is returned or an existing set is updated.

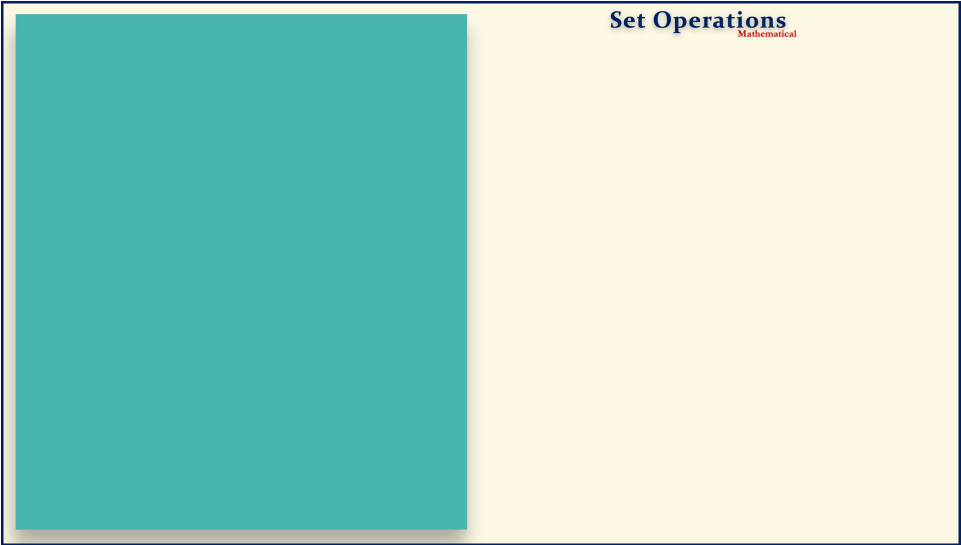
Section 4: Key Takeaways

- Python `set` methods mirror **mathematical set operations**: union, intersection, difference, symmetric difference.
- Each operation has two variants: one returning a **new set** and one performing an **in-place update**.
- Using **union** combines all unique elements, **intersection** finds common elements, **difference** subtracts elements from one set not found in the other, and **symmetric difference** gives unique elements across both sets.
- Update variants like `intersection\_update`, `difference\_update`, and `symmetric\_difference\_update` modify the **calling set directly**, useful for memory-efficient operations.
- Understanding these operations is key for working with collections that require uniqueness and set-based logic in Python programs.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.





Set Operations

Mathematical

union(iterable)

Set Operations

Mathematical

union(iterable)

intersection(iterable)

Set Operations

Mathematical

union(iterable)

intersection(iterable)

intersection_update(iterable)

Set Operations

Mathematical

union(iterable)

intersection(iterable)

intersection_update(iterable)

difference(iterable)



Set Operations

Mathematical

- `union( iterable )`
- `intersection( iterable )`
- `intersection_update( iterable )`
- `difference( iterable )`
- `difference_update( iterable )`



Set Operations

Mathematical

- `union( iterable )`
- `intersection( iterable )`
- `intersection_update( iterable )`
- `difference( iterable )`
- `difference_update( iterable )`
- `symmetric_difference( iterable )`

Set Operations

Mathematical

- `union( iterable )`
- `intersection( iterable )`
- `intersection_update( iterable )`
- `difference( iterable )`
- `difference_update( iterable )`
- `symmetric_difference( iterable )`
- `symmetric_difference_update( iterable )`

Set Operations

Mathematical

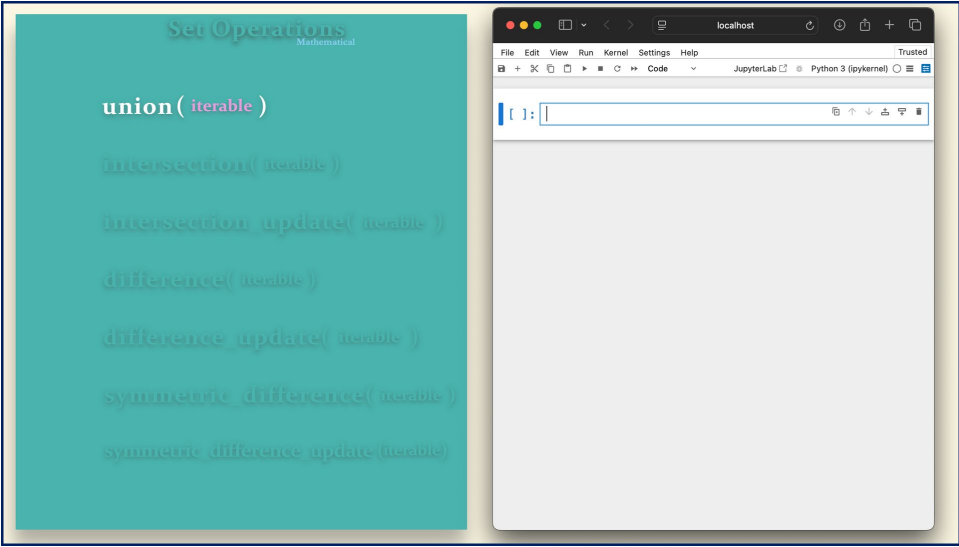
- `union( iterable )`
- `intersection( iterable )`
- `intersection_update( iterable )`
- `difference( iterable )`
- `difference_update( iterable )`
- `symmetric_difference( iterable )`
- `symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

`In [ ]:`



Set Operations

Mathematical

union(iterable)

intersection(iterable)

intersection_update(iterable)

difference(iterable)

difference_update(iterable)

symmetric_difference(iterable)

symmetric_difference_update(iterable)

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: s1={1,2,3,5,7}

[]:

S1

1 2 3 5 7

Set Operations

Mathematical

union(iterable)

intersection(iterable)

intersection_update(iterable)

difference(iterable)

difference_update(iterable)

symmetric_difference(iterable)

symmetric_difference_update(iterable)

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: s1={1,2,3,5,7}

[2]: s2={5,7,9,10,11}

[]:

S1

1 2 3 5 7

S2

5 7 9 10 11

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s1={1,2,3,5,7}
[2]: s2={5,7,9,10,11}
[ ]: 
```

S1

1 2 3 5 7

S2

5 7 9 10 11

`s3 = s1.union( s2 )`

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s1={1,2,3,5,7}
[2]: s2={5,7,9,10,11}
[3]: s3=s1.union(s2)
[ ]: 
```

S1

1 2 3 5 7

S2

✗ ✗ 9 10 11

S3

1 2 3 5 7 9 10 11

`s3 = s1.union( s2 )`

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s1={1,2,3,5,7}
[2]: s2={5,7,9,10,11}
[3]: s3=s1.union(s2)
[4]: print(s3)
{1, 2, 3, 5, 7, 9, 10, 11}
```

S1

12357

S2

1

2

3

5

7

9

10

11

S3

1235791011

`s3 = s1.union( s2 )`

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
[ ]:
```

Set Operations

Mathematical

union(iterable)

intersection(iterable)

intersection_update(iterable)

difference(iterable)

difference_update(iterable)

symmetric_difference(iterable)

symmetric_difference_update(iterable)

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: s1={1,2,3,5,7}

[]:

S1 1 2 3 5 7

Set Operations

Mathematical

union(iterable)

intersection(iterable)

intersection_update(iterable)

difference(iterable)

difference_update(iterable)

symmetric_difference(iterable)

symmetric_difference_update(iterable)

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: s1={1,2,3,5,7}

[2]: s2={5,7,9,10,11}

[]:

S1 1 2 3 5 7 S2 5 7 9 10 11

Set Operations

Mathematical

union(iterable)

intersection(iterable)

intersection_update(iterable)

difference(iterable)

difference_update(iterable)

symmetric_difference(iterable)

symmetric_difference_update(iterable)

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: s1={1,2,3,5,7}

[2]: s2={5,7,9,10,11}

{ }:

S1 1 2 3 5 7 S2 5 7 9 10 11

s3 = s1.intersection(s2)

Set Operations

Mathematical

union(iterable)

intersection(iterable)

intersection_update(iterable)

difference(iterable)

difference_update(iterable)

symmetric_difference(iterable)

symmetric_difference_update(iterable)

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: s1={1,2,3,5,7}

[2]: s2={5,7,9,10,11}

{ }:

S1 1 2 3 5 7 S2 5 7 9 10 11

s3 = s1.intersection(s2)

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

```
[1]: s1={1,2,3,5,7}
[2]: s2={5,7,9,10,11}
[3]: s3=s1.intersection(s2)
[ ]:
```

S1 1 2 3 5 7 S2 5 7 9 10 11

S3 5 7

`s3 = s1.intersection( s2 )`

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s1={1,2,3,5,7}
[2]: s2={5,7,9,10,11}
[3]: s3=s1.intersection(s2)
[4]: print(s3)
{5, 7}
```

s1

12357

s2

5791011

s3

57

`s3 = s1.intersection( s2 )`

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```
[ ]:
```

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: `s1={1,2,3,5,7}`
`s2={5,7,9,10,11}`

{ }:

S1

1 2 3 5 7

S2

5 7 9 10 11

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: `s1={1,2,3,5,7}`
`s2={5,7,9,10,11}`

{ }:

S1

1 2 3 5 7

S2

5 7 9 10 11

`s1.intersection_update( s2 )`

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: `s1={1,2,3,5,7}`
`s2={5,7,9,10,11}`

[]:

S1

1 2 3 5 7

S2

5 7 9 10 11

`s1.intersection_update( s2 )`

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: `s1={1,2,3,5,7}`
`s2={5,7,9,10,11}`

[2]: `s1.intersection_update(s2)`

[]:

S1

5 7

S2

5 7 9 10 11

`s1.intersection_update( s2 )`

Set Operations

Mathematical

union(iterable)

intersection(iterable)

intersection_update(iterable)

difference(iterable)

difference_update(iterable)

symmetric_difference(iterable)

symmetric_difference_update(iterable)

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s1={1,2,3,5,7}
s2={5,7,9,10,11}
[2]: s1.intersection_update(s2)
[3]: print(s1)
{5, 7}
```

S1

57

S2

5791011

s1.intersection_update(s2)

Set Operations

Mathematical

union(iterable)

intersection(iterable)

intersection_update(iterable)

difference(iterable)

difference_update(iterable)

symmetric_difference(iterable)

symmetric_difference_update(iterable)

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
[ ]:
```

Set Operations

Mathematical

union(iterable)

intersection(iterable)

intersection_update(iterable)

difference(iterable)

difference_update(iterable)

symmetric_difference(iterable)

symmetric_difference_update(iterable)

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: s1={1,2,3,5,7}
s2={5,7,9,10,11}

[]:

S1 1 2 3 5 7 S2 5 7 9 10 11

Set Operations

Mathematical

union(iterable)

intersection(iterable)

intersection_update(iterable)

difference(iterable)

difference_update(iterable)

symmetric_difference(iterable)

symmetric_difference_update(iterable)

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: s1={1,2,3,5,7}
s2={5,7,9,10,11}

[]:

S1 1 2 3 5 7 S2 5 7 9 10 11

s3 = s1.difference(s2)

Set Operations

Mathematical

- `union( iterable )`
- `intersection( iterable )`
- `intersection_update( iterable )`
- `difference( iterable )`**
- `difference_update( iterable )`
- `symmetric_difference( iterable )`
- `symmetric_difference_update( iterable )`

```
[1]: s1={1,2,3,5,7}
     s2={5,7,9,10,11}
```

```
[ ]:
```

S1

12357

S2

5791011

`s3 = s1.difference( s2 )`

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

```
[1]: s1={1,2,3,5,7}
      s2={5,7,9,10,11}

[2]: s3=s1.difference(s2)

In [ ]:
```

S1

12357

S2

5791011

S3

123

`s3 = s1.difference( s2 )`

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

```
[1]: s1={1,2,3,5,7}
      s2={5,7,9,10,11}

[2]: s3=s1.difference(s2)

[3]: print(s3)
      {1, 2, 3}

In [ ]:
```

S1

12357

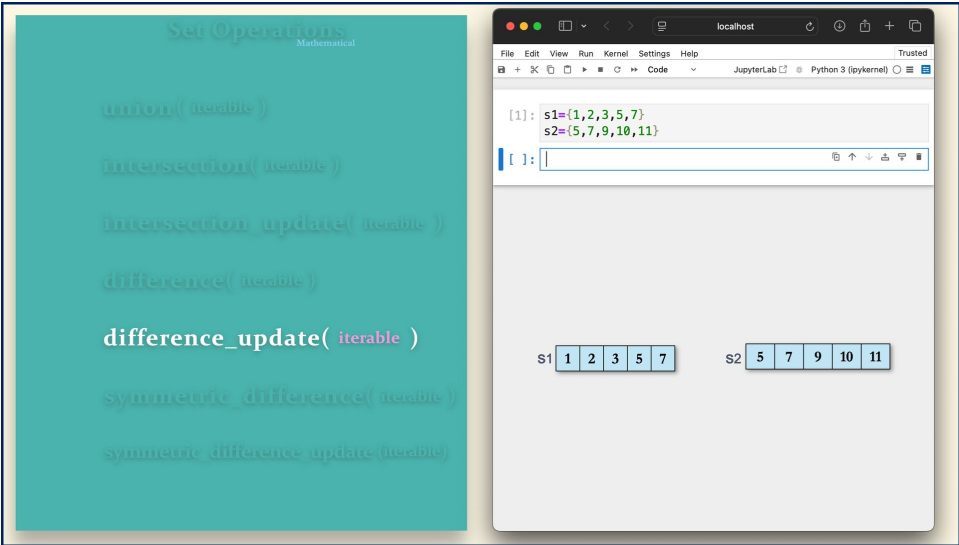
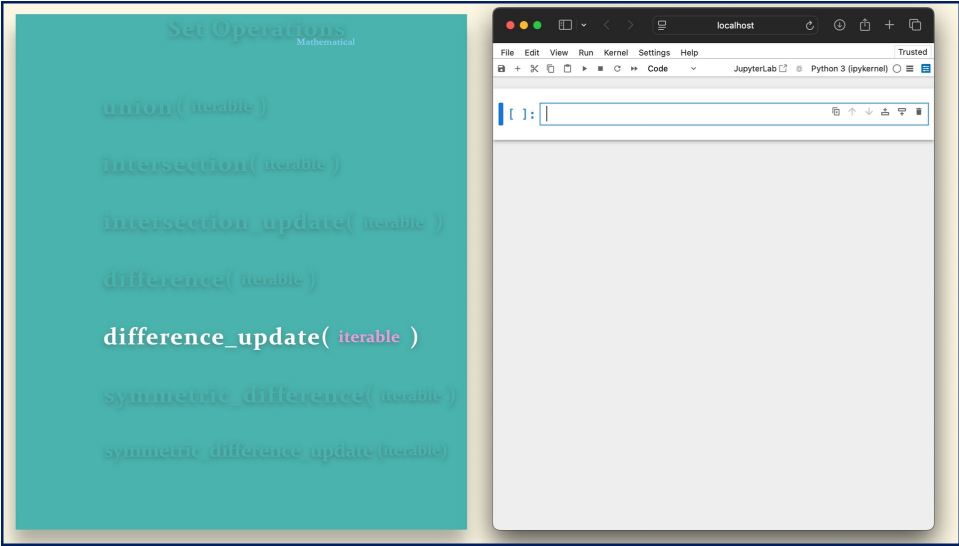
S2

5791011

S3

123

`s3 = s1.difference( s2 )`



Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: `s1={1,2,3,5,7}`
`s2={5,7,9,10,11}`

[]:

S1

1 2 3 5 7

S2

5 7 9 10 11

`s1.difference_update( s2 )`

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: `s1={1,2,3,5,7}`
`s2={5,7,9,10,11}`

[]:

S1

1 2 3 5 7

S2

5 7 9 10 11

`s1.difference_update( s2 )`

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s1={1,2,3,5,7}
      s2={5,7,9,10,11}
[2]: s1.difference_update(s2)
```

```
{ 1: |
```

S1

1 2 3

S2

5 7 9 10 11

`s1.difference_update( s2 )`

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s1={1,2,3,5,7}
      s2={5,7,9,10,11}
[2]: s1.difference_update(s2)
[3]: print(s1)
```

```
{1, 2, 3}
```

```
{ 1: |
```

S1

1 2 3

S2

5 7 9 10 11

`s1.difference_update( s2 )`

Set Operations

Mathematical

union(iterable)

intersection(iterable)

intersection_update(iterable)

difference(iterable)

difference_update(iterable)

symmetric_difference(iterable)

symmetric_difference_update(iterable)

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: s1={1,2,3,5,7}
s2={5,7,9,10,11}

[]:

S1 1 2 3 5 7 S2 5 7 9 10 11

s2.difference_update(s1)

Set Operations

Mathematical

union(iterable)

intersection(iterable)

intersection_update(iterable)

difference(iterable)

difference_update(iterable)

symmetric_difference(iterable)

symmetric_difference_update(iterable)

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: s1={1,2,3,5,7}
s2={5,7,9,10,11}

[2]: s2.difference_update(s1)

[]:

S1 1 2 3 5 7 S2 9 10 11

s2.difference_update(s1)

Set Operations

Mathematical

- `union( iterable )`
- `intersection( iterable )`
- `intersection_update( iterable )`
- `difference( iterable )`
- `difference_update( iterable )`**
- `symmetric_difference( iterable )`
- `symmetric_difference_update( iterable )`

```
[1]: s1={1,2,3,5,7}
     s2={5,7,9,10,11}

[2]: s2.difference_update(s1)

[3]: print(s2)
{9, 10, 11}
```

`{ 1: |`

S1

12357

S2

91011

`s2.difference_update( s1 )`

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
[ ]:
```

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s1={1,2,3,5,7}
      s2={5,7,9,10,11}
```

[]:

s1

12357

s2

5791011

Set Operations

Mathematical

union(iterable)

intersection(iterable)

intersection_update(iterable)

difference(iterable)

difference_update(iterable)

symmetric_difference(iterable)

symmetric_difference_update(iterable)

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: s1={1,2,3,5,7}
s2={5,7,9,10,11}

{ }:

S1 1 2 3 5 7 S2 5 7 9 10 11

s3 = s1.symmetric_difference(s2)

Set Operations

Mathematical

union(iterable)

intersection(iterable)

intersection_update(iterable)

difference(iterable)

difference_update(iterable)

symmetric_difference(iterable)

symmetric_difference_update(iterable)

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: s1={1,2,3,5,7}
s2={5,7,9,10,11}

{ }:

S1 1 2 3 5 7 S2 5 7 9 10 11

s3 = s1.symmetric_difference(s2)

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s1={1,2,3,5,7}
      s2={5,7,9,10,11}
[2]: s3=s1.symmetric_difference(s2)
      [ ]:
```

S1

1 2 3 5 7

S2

5 7 9 10 11

S3

1 2 3 9 10 11

`s3 = s1.symmetric_difference( s2 )`

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s1={1,2,3,5,7}
      s2={5,7,9,10,11}
[2]: s3=s1.symmetric_difference(s2)
[3]: print(s3)
      {1, 2, 3, 9, 10, 11}
      [ ]:
```

S1

1 2 3 5 7

S2

5 7 9 10 11

S3

1 2 3 9 10 11

`s3 = s1.symmetric_difference( s2 )`

Set Operations

Mathematical

- `union( iterable )`
- `intersection( iterable )`
- `intersection_update( iterable )`
- `difference( iterable )`
- `difference_update( iterable )`
- `symmetric_difference( iterable )`
- `symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
[ ]:
```

Set Operations

Mathematical

- `union( iterable )`
- `intersection( iterable )`
- `intersection_update( iterable )`
- `difference( iterable )`
- `difference_update( iterable )`
- `symmetric_difference( iterable )`
- `symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s1={1,2,3,5,7}
      s2={5,7,9,10,11}
```

[]:

s1

12357

s2

5791011

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]:

`s1={1,2,3,5,7}`
`s2={5,7,9,10,11}`

{ }:

s1

1 2 3 5 7

s2

5 7 9 10 11

`s1.symmetric_difference_update( s2 )`

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]:

`s1={1,2,3,5,7}`
`s2={5,7,9,10,11}`

{ }:

s1

1 2 3 5 7

s2

5 7 9 10 11

`s1.symmetric_difference_update( s2 )`

Set Operations

Mathematical

- `union( iterable )`
- `intersection( iterable )`
- `intersection_update( iterable )`
- `difference( iterable )`
- `difference_update( iterable )`
- `symmetric_difference( iterable )`
- `symmetric_difference_update( iterable )`

```
[1]: s1={1,2,3,5,7}
      s2={5,7,9,10,11}

[2]: s1.symmetric_difference_update(s2)

[ ]:
```

s1

12391011

s2

5791011

`s1.symmetric_difference_update( s2 )`

Set Operations

Mathematical

`union( iterable )`

`intersection( iterable )`

`intersection_update( iterable )`

`difference( iterable )`

`difference_update( iterable )`

`symmetric_difference( iterable )`

`symmetric_difference_update( iterable )`

```
localhost
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

[1]: s1={1,2,3,5,7}
      s2={5,7,9,10,11}

[2]: s1.symmetric_difference_update(s2)

[3]: print(s1)

{1, 2, 3, 9, 10, 11}

[ ]: |

s1 1 2 3 9 10 11 s2 5 7 9 10 11

s1.symmetric_difference_update( s2 )
```